Kubernetes

requests / limitsでリソース暴走を防ぐ

コンテナのリソース制限と安定稼働の設計

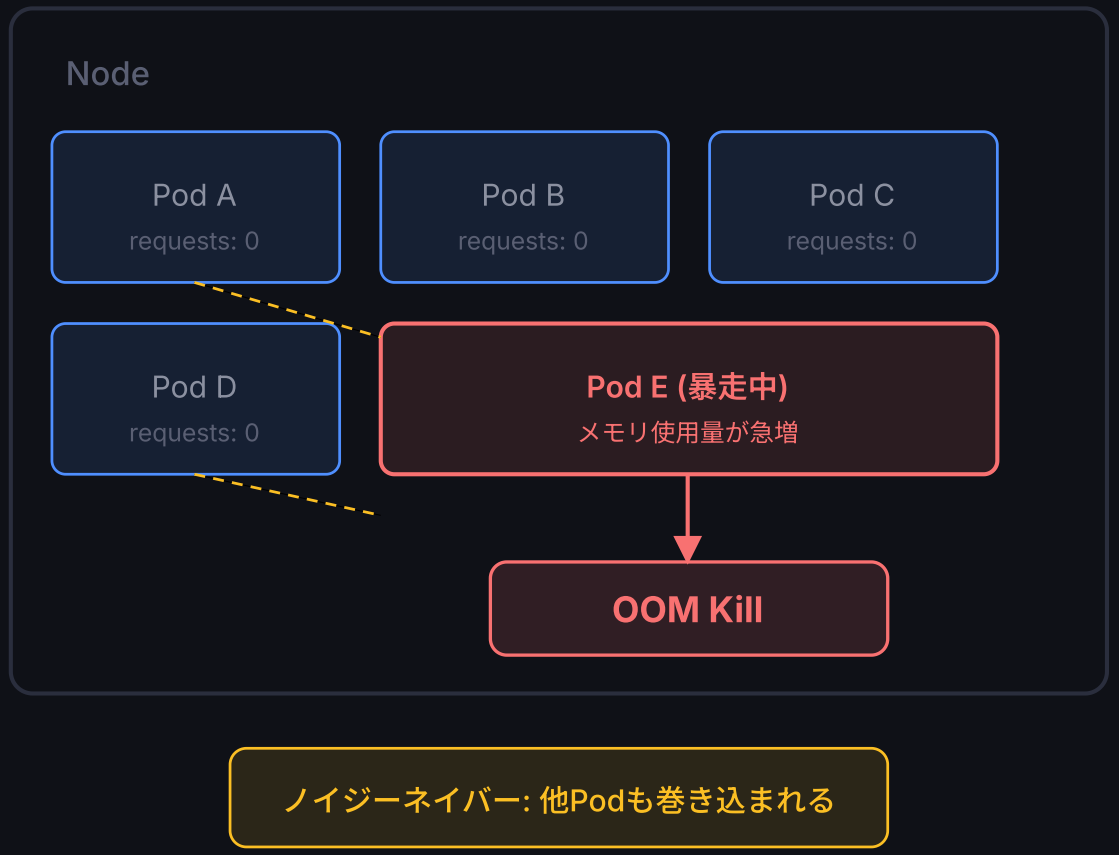
viserhaut — 2026

SECTION 01

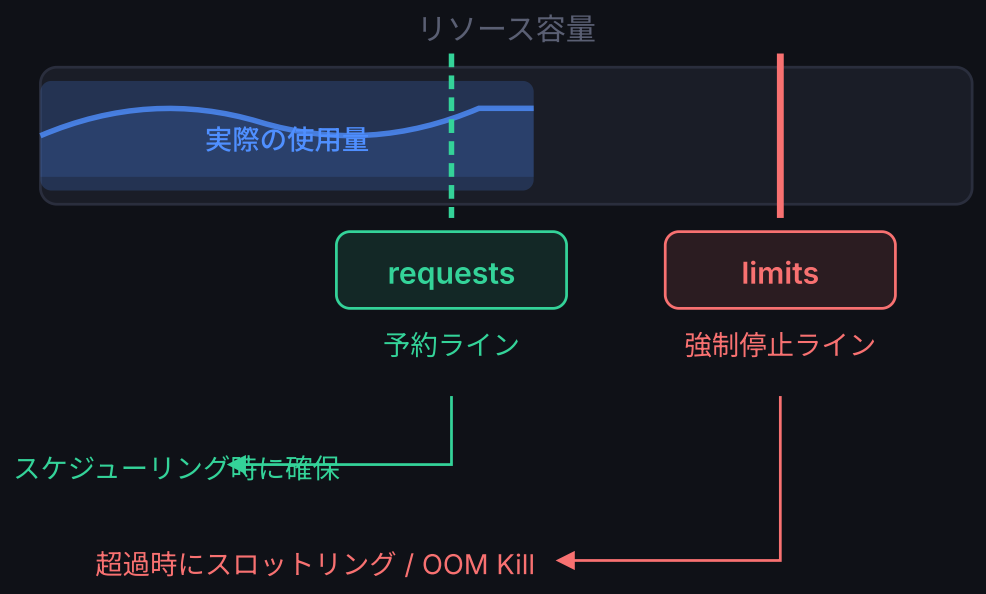
リソース制限が必要な理由

設定なしではノード過剰配置とOOM Killが発生する

- requestsがゼロだとKubernetesが「リソース不要」と判断し、Nodeに**Podが過剰配置**される
- limitsがないとメモリ暴走時に**OOM Killer**がPodを強制終了する
- 1つのPodの暴走がNode全体を巻き込む**ノイジーネイバー問題**が発生する



requestsで予約しlimitsで制限する2段階の仕組み



- requestsは「最低限確保するリソース量」で、スケジューリング判断に使われる
- limitsは「絶対に超えてはいけない上限」で、実行中のリソース制御に使われる
- requestsは**予約**、limitsは**強制ストップライン**と覚える

requests vs limits — 役割・使われ方・未設定リスクの比較

観点	REQUESTS	LIMITS
役割	最低限確保するリソース量	最大使用可能なリソース量
使われ方	スケジューリング判断に使用	実行中の制御に使用
未設定リスク	Nodeにpodが過剰配置されパンクする	リソース暴走が止められずNode障害に

resources設定の実践マニフェスト

```yaml
containers:
- name: api
  resources:
    requests:
      memory: "128Mi"
      cpu: "250m"
    limits:
      memory: "256Mi"
      cpu: "500m"
```

requests
CPU 250m / メモリ128MiをPodに最低保証する

limits
CPU 500m / メモリ256Miを超えると制限される

設計指針
limitsはrequestsの1.5〜2倍が目安

スケジューリングからOOM Killまでのリソース制御フロー

本番運用でのリソース設計3原則

01

requests / limitsは必ずセットで設定する

片方だけでは過剰配置や暴走を防げない。両方設定が安全運用の第一歩

02

実測値を元にrequestsを決める

kubectl topで実際の使用量を確認し、適切なrequests値を設定する

03

limitsはrequestsの1.5〜2倍に設定する

バースト的な負荷に対応しつつ、暴走は確実に止める余裕を持たせる